АСЭ
РОСАТОМ

Лекция 1

Vite, Vite+, Oxc и Nginx

Эрнандес Гарсия Даниэль Дельевич

Веб-разработчик

О чём лекция

Соберём инструментарий, с которым фронтенд-проект удобно запускать, проверять и доставлять до пользователя.

Vite отвечает за разработку и сборку: мгновенный старт, HMR и оптимизированный production-бандл.

Vite+ добавляет единый CLI: зависимости, линтинг, форматирование, тесты и задачи.

Oxc даёт быстрые линтер и форматтер на Rust.

Nginx принимает трафик в проде: статика, SPA-fallback, proxy и HTTPS.

Разберём, как всё устроено, какие настройки действительно важны и как выглядит рабочий конфиг.

АСЭ
РОСАТОМ

01

Vite

Dev-сервер, сборка и конфиг

Как Vite ускоряет разработку

В dev-режиме Vite не собирает весь проект заранее. Он отдаёт исходники по запросу как ESM: браузер сам подгружает модули, а Vite пересобирает только изменённое. Поэтому старт почти мгновенный, а HMR обновляет страницу без полной перезагрузки.

В production Vite собирает оптимизированный бандл: минифицирует код, разбивает его на чанки и подготавливает ассеты.

Команда	Что делает
npm create vite@latest	Новый проект из шаблона
npm install -D vite	Vite в существующий проект

Шаблоны: react, react-ts, vue, svelte, solid, preact, lit, vanilla.

Конфиг: файл и форма

Конфиг Vite живёт в vite.config.ts. Минимальный вариант выглядит так:

```
import { defineConfig } from 'vite';

export default defineConfig({});
```

Если настройки зависят от режима, экспортируется функция. Vite передаёт в неё command: serve при запуске vite и build при vite build. По этому значению можно включать разные плагины и настройки.

```
export default defineConfig(({ command }) => ({
  base: command === 'build' ? '/app/' : '/',
}));
```

.env в конфиге

Кажется логичным прочитать переменные через process.env прямо в vite.config.ts. Но Vite загружает .env позже: набор файлов зависит от самого конфига, поэтому на момент выполнения конфига их ещё нет. В process.env доступны только переменные окружения операционной системы.

process.env в Node.js — глобальный объект с переменными окружения процесса, а process.cwd() возвращает текущий рабочий каталог.

Если .env всё-таки нужен в конфиге, его загружают вручную функцией loadEnv:

```
import { defineConfig, loadEnv } from 'vite';

export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), '');
  return {
    define: { __API__: JSON.stringify(env.VITE_API_URL) },
  };
});
```

root и base: где проект и куда деплоить

АСЭ
РОСАТОМ

root указывает папку, в которой лежит index.html. По умолчанию это process.cwd() — каталог, из которого вы запустили команду. Если index.html находится в подпапке, без явного root dev-сервер его не найдёт.

base нужен, когда приложение живёт не в корне домена, а в подпапке. В сборке он подставляет префикс во все пути и задаёт import.meta.env.BASE_URL; в dev-сервере приложение открывается по тому же пути, например localhost:5173/app/.

Опция	По умолчанию	Когда менять
root	process.cwd()	index.html в подпапке
base	'/'	Деплой в подпапку

define: подстановка значений

АСЭ
РОСАТОМ

define заменяет идентификаторы в коде на этапе сборки — примерно как #define в C. В dev-режиме значение просто добавляется в globalThis, а в сборке Vite меняет код на уровне AST и подставляет значение вместо идентификатора.

Главная ловушка — строки. Значение подставляется как есть, поэтому строку нужно оборачивать в JSON.stringify, иначе в коде окажется не строка, а идентификатор. Чтобы TypeScript знал о глобалах, их объявляют в vite-env.d.ts.

```
define: {
  __APP_VERSION__: JSON.stringify('1.0.0'),
}
```

plugins и publicDir

plugins — массив плагинов Vite. false, null и undefined просто игнорируются, поэтому удобно писать isDev && plugin(). Вложенные массивы Vite распаковывает, а если плагин возвращает Promise, дожидается его разрешения перед запуском.

publicDir — папка статичных файлов, которые отдаются как есть: favicon, robots.txt, картинки без обработки. По умолчанию это public. В dev-режиме файлы доступны по корню /, а в сборке копируются в корень outDir (dist). Если статика не нужна, publicDir: false отключает и обслуживание, и копирование.

В plugins	Поведение
false / null / undefined	игнорируется
вложенный массив	распаковывается
Promise	дожидается разрешения

resolve и служебные опции

resolve.extensions позволяет опускать расширения в import. По умолчанию список такой: .mjs, .js, .mts, .ts, .jsx, .tsx, .json. Если подходит несколько файлов, побеждает расширение, которое стоит в списке раньше.

resolve.tsconfigPaths включает алиасы из compilerOptions.paths в tsconfig.json. По умолчанию опция выключена.

clearScreen очищает терминал (по умолчанию true). envDir — папка с .env (по умолчанию root; false отключает загрузку). envPrefix — префикс клиентских переменных (по умолчанию VITE_): его меняют, чтобы серверные секреты не попали в бандл.

Опция	По умолчанию	Зачем
resolve.extensions	список расширений	Опускать расширения
resolve.tsconfigPaths	false	Алиасы из tsconfig
clearScreen	true	Чистый терминал
envDir	root	Папка с .env
envPrefix	VITE_	Клиентские переменные

Dev-сервер: порт, HTTPS и proxy

По умолчанию dev-сервер занимает порт 5173. Если порт занят, Vite попробует следующий; strictPort запрещает это и просто не запускается — удобно, когда порт важен для внешних интеграций.

https включает TLS/HTTP2 для dev-сервера. Можно передать { key, cert } или подключить плагин @vitejs/plugin-basic-ssl. open открывает браузер при старте, а строковое значение open задаёт путь, который нужно открыть.

cors настраивает CORS dev-сервера: true разрешает любой origin, объект — как CORS-middleware. proxy перенаправляет запросы на бэкенд и обходит CORS. forwardConsole: true отправляет консоль браузера в терминал Vite — полезно AI-агентам, которым нужно видеть ошибки клиента.

Опция	По умолчанию	Что делает
port	5173	Порт dev-сервера
strictPort	false	Запрещает другой порт
https	false	TLS/HTTP2
open	false	Открывает браузер
cors	false	CORS
proxy	—	Прокси на бэкенд
forwardConsole	false	Консоль в терминал

Плагины и кеш зависимостей

Для React достаточно @vitejs/plugin-react. Если нужен HTTPS в разработке, подключается @vitejs/plugin-basic-ssl:

```
import basicSsl from '@vitejs/plugin-basic-ssl';

export default {
  plugins: [basicSsl()],
};
```

@vitejs/plugin-react включает Fast Refresh: компоненты обновляются без потери состояния. basicSsl выпускает самоподписанный сертификат для локального HTTPS.

Предварительно упакованные зависимости Vite кеширует в node_modules/.vite — это ускоряет повторный старт.

Кеш можно удалить, если зависимости обновились, а dev-сервер ведёт себя странно: при следующем запуске Vite соберёт его заново.

Ассеты и стили

С ассетами всё просто: import imgUrl from './img.png' при сборке превращается в URL файла. Дальше imgUrl используется как обычная ссылка, например в src тега <img>.

Для .scss, .sass, .less и .styl отдельные плагины не нужны — поддержка встроена. Достаточно поставить сам препроцессор как dev-зависимость:

```
npm install -D sass
```

Vite сам подхватит препроцессор и будет учитывать алиасы из своего конфига в @import для Sass/Less.

.env и режимы: что и когда загружается

В клиентский код попадают только переменные с префиксом VITE_. Все значения в import.meta.env — строки, даже если в .env записано число или булево, поэтому сравнивать нужно явно: import.meta.env.VITE_F === 'true'. Без префикса переменная вернёт undefined.

Файлы загружаются по приоритету:

Приоритет	Файл
1	.env.[mode].local
2	.env.[mode]
3	.env.local
4	.env

Встроенные поля: MODE — текущий режим (development или production), BASE_URL — базовый URL из base, PROD/DEV/SSR — булевы флаги продакшена, разработки и серверного рендеринга. Кастомные переменные всегда строки, встроенные — boolean.

Типы для import.meta.env

АСЭ
РОСАТОМ

import.meta — объект с метаданными текущего ESM-модуля. Vite добавляет в него env — объект с переменными окружения. В TypeScript его структуру описывает интерфейс ImportMetaEnv.

По умолчанию TypeScript знает только встроенные поля: MODE, DEV, PROD, BASE_URL, SSR. На кастомную переменную без типизации он ругается. Решение — расширить интерфейсы в src/vite-env.d.ts.

Директива /// <reference types="vite/client" /> подключает типы встроенных полей и ассетов.

```
/// <reference types="vite/client" />

interface ImportMetaEnv {
  readonly VITE_APP_TITLE: string;
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

АСЭ
РОСАТОМ

02

Vite+

Единый CLI для зависимостей и проверок

Vite+: один CLI вместо зоопарка

Vite+ — это глобальный CLI vp и локальный пакет vite-plus в каждом проекте. Конфиг остаётся стандартным vite.config.ts, поэтому переход не ломает привычную настройку.

Основные команды: vp dev запускает dev-сервер, vp build собирает проект, vp upgrade обновляет глобальный CLI. Для CI используют voidzero-dev/setup-vp.

Зависимости ставятся через vp:

Команда	Действие
vp add <pkg>	добавить пакет
vp add -D <pkg>	добавить dev-пакет
vp remove <pkg>	удалить пакет
vp update	обновить зависимости

Менеджер пакетов задаётся полем devEngines.packageManager в package.json, а точная версия — полем packageManager, например bun@1.1.20.

Проверки: check и lint

vp check — одна команда для форматирования, линтинга и проверки типов; флаг --fix включает автоисправление. Под капотом vp lint работает на Oxlint. У него есть typeCheck (запускает компилятор TypeScript во время линтинга) и typeAware (ищет логические ошибки на основе типов).

Конфиг линтера живёт в vite.config.ts, в блоке lint, а не в oxlint.config.ts:

```
export default defineConfig({
  lint: { ignorePatterns: ['dist/**'] },
});
```

vp lint запускает линтинг, vp lint --fix — с автоисправлением. Флаг --fix у vp check включает автоисправление сразу для форматтера и линтера.

Форматирование и тесты

АСЭ
РОСАТОМ

vp fmt форматирует код. По умолчанию он ничего не пишет в файлы: чтобы сохранить результат, нужно vp fmt . --write. Под капотом — Oxfmt.

Тесты запускаются командой vp test и работают на Vitest.

Оба конфига живут в vite.config.ts — в блоках fmt и test, а не в отдельных oxfmt.config.ts и vitest.config.ts.

Команда	Что делает
vp fmt	форматирование
vp fmt . --write	записать в файлы
vp test	тесты (Vitest)

Инструмент	Блок	Не в файле
fmt	fmt	oxfmt.config.ts
test	test	vitest.config.ts

Задачи и vpx

vp run <script-name> запускает скрипт из package.json. Разница с vp dev: vp dev — встроенная команда, а vp run dev запускает именно скрипт dev из package.json.

Задача — именованный скрипт. Задачи описываются в vite.config.ts, в блоке run.tasks; у задачи есть command и dependsOn — зависимость от другой задачи. Vite+ кеширует состояние задачи по входным и выходным файлам, поэтому повторный запуск может быть почти мгновенным.

```
run: {
  tasks: {
    build: { command: 'vp build', dependsOn: ['test'] },
  },
}
```

Запуск задачи — vp run <task-name>. Для разовых бинарников есть vpx — аналог npx и bunx.

Git-хуки

Хуки включаются командой vp hooks enable и выключаются через vp hooks disable; vp hooks status показывает текущее состояние.

Скрипты лежат в папке .vite-hooks в корне проекта. Её нужно коммитить, чтобы у всей команды были одинаковые хуки. Для pre-commit создаётся файл .vite-hooks/pre-commit с командой vp staged — она проверяет только изменённые файлы.

```
# .vite-hooks/pre-commit
vp staged
```

Через staged в vite.config можно задать свои команды: staged: { '*.{js,ts,tsx}': 'vp check --fix' }.

АСЭ
РОСАТОМ

03

Oxc

Линтер и форматтер на Rust

Oxc и Oxlint

Oxc — набор высокопроизводительных инструментов для JavaScript и TypeScript, написанных на Rust. Oxlint — его линтер на стеке Oxc: быстрый, совместимый с ESLint и использующий tsgo (TypeScript 7).

Установка и скрипты:

```
npm install -D oxlint


// package.json
"scripts": {
  "lint": "oxlint",
  "lint:fix": "oxlint --fix"
}
```

Oxlint проверяет код, а --fix автоматически исправляет то, что можно исправить. Запуск: npm run lint или npm run lint:fix.

Конфиг и плагины Oxlint

Конфиг создаётся командой oxlint --init. Файл называется .oxlintrc.json или oxlint.config.ts. Базовый вариант:

```
import { defineConfig } from 'oxlint';

export default defineConfig({});
```

Плагин — это именованная группа правил; его включение делает правила доступными. Встроенные плагины перечисляются в поле plugins, например plugins: ['import']. Эта запись переопределяет набор плагинов, включённых по умолчанию. Плагин eslint включён по умолчанию и подключает основные правила ESLint.

АСЭ
РОСАТОМ

04

Nginx

Точка входа в production: статика, proxy и HTTPS

Зачем Nginx фронтенду

После сборки Vite получается папка dist с HTML, JS, CSS и шрифтами. Отдавать её через Node-сервер можно, но дороже и медленнее: Nginx делает это эффективнее и берёт на себя инфраструктурные задачи.

Статика — быстрая отдача файлов.

Reverse proxy — прячет API, снимает CORS и балансирует нагрузку между инстансами.

TLS и кэш — терминирует HTTPS, сжимает gzip и кэширует ассеты на клиенте.

В типичном деплое Nginx — точка входа, а Node/API — за ним.

Роль	Что делает
Статика	HTML, JS, CSS, шрифты
Reverse proxy	API, CORS, балансировка
TLS и кэш	HTTPS, gzip, кэш

Карта конфига и ключевые директивы

Конфиг Nginx вложен сверху вниз: настройки из http наследуются в server, а из server — в location. Простые директивы заканчиваются ;, блочные — { }. Подключение файлов — include /etc/nginx/conf.d/*.conf;.

Контекст	Что настраивает
main	user, worker_processes, error_log
events	worker_connections
http	mime.types, gzip, upstream
server	listen, server_name, root
location	try_files, proxy_pass

Для SPA важны try_files (fallback на index.html), для API — proxy_pass, для ассетов — expires и gzip, для HTTPS — ssl и return 301.

Минимальный конфиг SPA

Вот минимальный server, которого хватает для SPA: статика, fallback на index.html и proxy на API.

```
server {
  listen 80;
  server_name example.com;
  root /usr/share/nginx/html;


  location / {
    try_files $uri $uri/ /index.html;
  }
  location /api/ { proxy_pass http://api:3000/; }
  location ~* \.(js|css|png|svg|woff2)$ {
    expires 30d;
    add_header Cache-Control "public, immutable";
  }
}
```

Без try_files прямой заход на /posts/1 даст 404. gzip, кэш и HTTPS добавляются в этот же server. Проверка: nginx -t → nginx -s reload.

Итог

Vite — быстрый dev-сервер и сборщик; его конфиг живёт в vite.config.ts.

Vite+ — единый CLI vp для зависимостей, проверок, тестов и задач.

Oxc — Oxlint и Oxfmt на Rust для быстрого линтинга и форматирования.

Nginx — точка входа в проде: статика, SPA-fallback, proxy и HTTPS.

Дальше: собрать проект на Vite+, настроить vp check и развернуть его за Nginx.